LLM Serving and Inference Optimization

Scaler School of Technology | ML System Design and LLMOps | Week 3 — Session 1/2 | Pre-Lecture Prep

What Is This Lecture About?

This session goes inside the inference engine. You have used GPT-4 via API, maybe fine-tuned a model. Now you will learn why deploying an LLM at scale is an engineering problem measured in hundreds of gigabytes of memory and thousands of dollars per day — and the specific techniques the industry uses to make it tractable.

Why LLM Inference Is Hard

GPT-3 has 175B parameters. At fp16 that is 350 GB of RAM just for weights — and generating a single token requires reading all of them from memory. Unlike classification models, LLM inference is sequential and memory-bandwidth bound, not compute-bound.

KV Cache and PagedAttention

Every attention layer stores Keys and Values for all previous tokens. This KV cache can reach 100 GB per active request. vLLM's PagedAttention applies the OS virtual memory insight to eliminate 90% of memory fragmentation, enabling 24× higher throughput on the same hardware.

Quantization: the "almost free" lunch

Reducing precision from fp16 to INT8 or INT4 cuts memory by 2–4× with surprisingly small quality loss. We cover GPTQ, AWQ, and GGUF — the tools that made running 70B models on consumer hardware possible.

Batching, Speculative Decoding, and Deployment Choices

Continuous batching boosts GPU utilization from 20% to 70%+. Speculative decoding uses a small draft model to propose tokens the large model verifies in parallel, giving 2–3× speedup. Finally: when to self-host vs. use an API, and how to calculate the break-even.

What to Expect in the Session

- Opening: estimate the memory cost of serving your last LLM project
- Walk-through of transformer attention and where cost comes from
- KV cache explained from first principles with concrete numbers
- PagedAttention and vLLM — the OS paging insight
- Precision table: fp32, fp16, INT8, INT4 with real model sizes
- GPTQ and AWQ: how post-training quantization actually works
- Static vs. continuous batching with GPU utilization comparison
- Speculative decoding: the algorithm step by step
- Self-host vs. API cost comparison at 100M tokens/day
- Two in-session quiz questions

How to Prepare

- **Cost estimate exercise:** Pick any LLM project you have worked on (or a hypothetical: a customer support bot, a coding assistant). Estimate: how many tokens per request? How many requests per day? At $0.002/1K tokens, what is the monthly cost? Bring your estimate to class.
- Recall how transformer self-attention works: the QKV matrices, the attention formula. We will not re-derive it, but understanding why K and V need to be stored is key.

- Optional: read the vLLM blog post (search "Efficient Memory Management for Large Language Model Serving with PagedAttention"). The first two pages are accessible without a deep ML background.

- Optional: install `llama.cpp` and run a Q4 model locally. This is not required but gives intuition for what quantization buys you in practice.
- No new coding environment needed for this session — we focus on architecture and trade-offs.

Duration: ∼2 hours **Prerequisites:** Transformers architecture, Week 1–2 of this course **Instructor:** Priyansh Saxena